

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Open Data Analysis

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO4	Assessment:	Tool 2– Open Data Analysis
Weightage:		Total Marks:	
CO4 Statement: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.			
Student Name:	Sania Herbert	Reg. No.:	192324062
Section / Batch:	2023/AI&DS	Date of Submission:	16.8.26
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I Sania Herbert(192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

1. Select any **open government dataset** and perform data exploration using importing, table functions, filtering, sorting, grouping, correlation analysis, and outlier detection. Present the results using suitable charts and publish the final analysis on an open-source platform.

```
2 # QUESTION 1: GOVERNMENT DATASET ANALYSIS
3 # =====
4 # COMMAND 2: Import libraries
5 import pandas as pd
6 import numpy as np
7 import matplotlib.pyplot as plt
8 import seaborn as sns
9 # COMMAND 3: Import Excel dataset
10 df = pd.read_excel(r"C:\Users\DELL\Downloads\Q1_Government_COVID19_Dataset.xlsx")
11 # COMMAND 4: Display first 5 records
12 print("FIRST 5 RECORDS")
13 print(df.head())
14 # COMMAND 5: Display last 5 records
15 print("\nLAST 5 RECORDS")
16 print(df.tail())
17 # COMMAND 6: Display number of rows and columns
18 print("\nDATASET SHAPE")
```

```
18 print("\nDATASET SHAPE")
19 print(df.shape)
20 # COMMAND 7: Display column names
21 print("\nCOLUMN NAMES")
22 print(df.columns.tolist())
23 # COMMAND 8: Display dataset information
24 print("\nDATASET INFORMATION")
25 print(df.info())
26 # COMMAND 9: Statistical summary
27 print("\nSTATISTICAL SUMMARY")
28 print(df.describe())
29 # COMMAND 10: Check missing values
30 print("\nMISSING VALUES")
31 print(df.isnull().sum())
32 # COMMAND 11: Check duplicate records
33 print("\nDUPLICATE RECORDS")
34 print(df.duplicated().sum())
```

```
35 # COMMAND 12: Check unique states
36 print("\nUNIQUE STATES")
37 print(df["State"].unique())
38 # COMMAND 13: Filter records
39 print("\nRECORDS WITH CONFIRMED CASES > 70000")
40 high_cases = df[df["Confirmed"] > 70000]
41 print(high_cases)
42
43 # COMMAND 14: Filter one state
44 print("\nMAHARASHTRA RECORDS")
45 maharashtra = df[df["State"] == "Maharashtra"]
46 print(maharashtra.head())
47
48 # COMMAND 15: Sort by confirmed cases
49 print("\nTOP 10 RECORDS BY CONFIRMED CASES")
50 sorted_data = df.sort_values(
51     by="Confirmed",
```

```
52     ascending=False
53 )
54 print(sorted_data.head(10))
55
56 # COMMAND 16: Group by state
57 print("\nSTATE-WISE SUMMARY")
58 state_summary = df.groupby("State").agg({
59     "Confirmed": "sum",
60     "Recovered": "sum",
61     "Deaths": "sum"
62 })
63
64 print(state_summary)
65
66 # COMMAND 17: Sort state summary
67 print("\nTOP STATES BY CONFIRMED CASES")
68 state_summary = state_summary.sort_values(
69     by="Confirmed"
```

```

69     by="Confirmed",
70     ascending=False
71 )
72
73 print(state_summary.head(10))
74
75 # COMMAND 18: Correlation analysis
76 print("\nCORRELATION MATRIX")
77
78 correlation = df[
79     ["Confirmed", "Recovered", "Deaths"]
80 ].corr()
81
82 print(correlation)
83

```

```

82 # COMMAND 19: Correlation heatmap
83 plt.figure(figsize=(8, 6))
84 sns.heatmap(
85     correlation,
86     annot=True,
87     cmap="coolwarm",
88     fmt=".2f"
89 )
90 plt.title("Correlation Between COVID-19 Indicators")
91 plt.tight_layout()
92 plt.show()
93
94 # COMMAND 20: Outlier detection using IQR
95 Q1 = df["Confirmed"].quantile(0.25)
96 Q3 = df["Confirmed"].quantile(0.75)
97
98 IQR = Q3 - Q1
99

```

```

98 IQR = Q3 - Q1
99 lower_limit = Q1 - 1.5 * IQR
100 upper_limit = Q3 + 1.5 * IQR
101 print("\nOUTLIER LIMITS")
102 print("Q1:", Q1)
103 print("Q3:", Q3)
104 print("IQR:", IQR)
105 print("Lower Limit:", lower_limit)
106 print("Upper Limit:", upper_limit)
107 # COMMAND 21: Identify outliers
108 outliers = df[
109     (df["Confirmed"] < lower_limit) |
110     (df["Confirmed"] > upper_limit)
111 ]
112 print("\nOUTLIER RECORDS")
113 print(outliers)
114 print("\nNUMBER OF OUTLIERS:", len(outliers))
115 # COMMAND 22: Outlier visualization

```

```

115 # COMMAND 22: Outlier visualization
116 plt.figure(figsize=(8, 5))
117 sns.boxplot(
118     x=df["Confirmed"]
119 )
120 plt.title("Confirmed Cases - Outlier Detection")
121 plt.xlabel("Confirmed Cases")
122 plt.show()
123 # COMMAND 23: Top 10 states bar chart
124 top10 = state_summary.head(10)
125
126 plt.figure(figsize=(10, 6))
127
128 top10["Confirmed"].plot(
129     kind="bar"
130 )
131

```

```

129 plt.title("Top 10 States by Confirmed COVID-19 Cases")
130 plt.xlabel("State")
131 plt.ylabel("Confirmed Cases")
132 plt.xticks(rotation=45)
133 plt.tight_layout()
134 plt.show()
135 # COMMAND 24: Confirmed vs Recovered chart
136 plt.figure(figsize=(10, 6))
137
138 top10[["Confirmed", "Recovered"]].plot(
139     kind="bar",
140     figsize=(10, 6)
141 )
142 plt.title("Confirmed vs Recovered Cases")
143 plt.xlabel("State")
144 plt.ylabel("Cases")

```

```

145 plt.xticks(rotation=45)
146 plt.tight_layout()
147 plt.show()
148 # COMMAND 25: Deaths by state
149 plt.figure(figsize=(10, 6))
150 top10["Deaths"].plot(
151     kind="bar"
152 )
153 plt.title("Deaths by State")
154 plt.xlabel("State")
155 plt.ylabel("Deaths")
156 plt.xticks(rotation=45)
157 plt.tight_layout()
158 plt.show()
159 # COMMAND 26: Time-based analysis
160 daily_cases = df.groupby("Date")["Confirmed"].sum()
161 plt.figure(figsize=(12, 6))
162 daily_cases.plot()

```

```

162 daily_cases.plot()
163 plt.title("Daily Confirmed Cases Trend")
164 plt.xlabel("Date")
165 plt.ylabel("Confirmed Cases")
166 plt.grid()
167 plt.tight_layout()
168 plt.show()
169 # COMMAND 27: Save state-wise analysis
170 state_summary.to_excel(
171     "Q1_State_Wise_Analysis.xlsx"
172 )
173 # COMMAND 28: Save outliers
174 outliers.to_excel(
175     "Q1_COVID_Outliers.xlsx",
176     index=False
177 )
178 print("\nQUESTION 1 ANALYSIS COMPLETED SUCCESSFULLY")

```

FIRST 5 RECORDS

	Date	State	Latitude	Longitude	Confirmed	Recovered	Deaths
0	2021-01-01	Maharashtra	19.0	72.8	90030	76545	2085
1	2021-01-02	Maharashtra	19.0	72.8	97253	77323	2401
2	2021-01-03	Maharashtra	19.0	72.8	94512	85607	1292
3	2021-01-04	Maharashtra	19.0	72.8	90460	75926	2175
4	2021-01-05	Maharashtra	19.0	72.8	99108	90351	1761

LAST 5 RECORDS

	Date	State	Latitude	Longitude	Confirmed	Recovered	Deaths
895	2021-03-27	Andhra Pradesh	16.5	80.6	24094	21924	511
896	2021-03-28	Andhra Pradesh	16.5	80.6	28394	26202	426
897	2021-03-29	Andhra Pradesh	16.5	80.6	24920	22001	614
898	2021-03-30	Andhra Pradesh	16.5	80.6	24657	20218	341

DATASET SHAPE
(900, 7)

COLUMN NAMES
['Date', 'State', 'Latitude', 'Longitude', 'Confirmed', 'Recovered', 'Deaths']

DATASET INFORMATION

<class 'pandas.DataFrame'>

RangeIndex: 900 entries, 0 to 899

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	Date	900 non-null	datetime64[us]

0	Date	900 non-null	datetime64[us]
1	State	900 non-null	str
2	Latitude	900 non-null	float64
3	Longitude	900 non-null	float64
4	Confirmed	900 non-null	int64
5	Recovered	900 non-null	int64

6	Deaths	900 non-null	int64
---	--------	--------------	-------

dtypes: datetime64[us](1), float64(2), int64(3), str(1)
memory usage: 49.3 KB
None

STATISTICAL SUMMARY

	Date	Latitude	Longitude	Confirmed	Recovered	Deaths
count	900	900.000000	900.000000	900.000000	900.000000	900.000000
mean	2021-02-14 12:00:00	19.940000	78.250000	47064.495556	40437.482222	874.187778
min	2021-01-01 00:00:00	10.000000	72.600000	16446.000000	12980.000000	242.000000
25%	2021-01-23 00:00:00	13.100000	75.800000	30864.000000	26043.500000	549.000000
50%	2021-02-14 12:00:00	20.800000	77.400000	41504.500000	35672.500000	775.000000
75%	2021-03-09 00:00:00	26.800000	80.600000	61599.750000	52737.500000	1110.000000
max	2021-03-31 00:00:00	28.600000	88.400000	120800.000000	112924.000000	2530.000000
std	NaN	6.286837	4.392487	20906.419922	18188.063270	425.084256

MISSING VALUES

Date	0
State	0
Latitude	0
Longitude	0
Confirmed	0
Recovered	0

```

UNIQUE STATES
<StringArray>
[  'Maharashtra',    'Tamil Nadu',    'Karnataka',    'Delhi',
   'Kerala',    'Uttar Pradesh',    'Gujarat',    'West Bengal',
   'Rajasthan', 'Andhra Pradesh']
Length: 10, dtype: str

```

RECORDS WITH CONFIRMED CASES > 70000

	Date	State	Latitude	Longitude	Confirmed	Recovered	Deaths
0	2021-01-01	Maharashtra	19.0	72.8	90030	76545	2085
1	2021-01-02	Maharashtra	19.0	72.8	97253	77323	2401
2	2021-01-03	Maharashtra	19.0	72.8	94512	85607	1292
3	2021-01-04	Maharashtra	19.0	72.8	90460	75926	2175
4	2021-01-05	Maharashtra	19.0	72.8	99108	90351	1761

...	...	...	...	...	...	...	...
368	2021-01-09	Kerala	10.0	76.3	80438	74748	1991
373	2021-01-14	Kerala	10.0	76.3	71246	63013	1774
374	2021-01-15	Kerala	10.0	76.3	72304	57498	1552
375	2021-01-16	Kerala	10.0	76.3	70119	58264	1262
416	2021-02-26	Kerala	10.0	76.3	74353	64254	1856

MAHARASHTRA RECORDS

	Date	State	Latitude	Longitude	Confirmed	Recovered	Deaths
0	2021-01-01	Maharashtra	19.0	72.8	90030	76545	2085
1	2021-01-02	Maharashtra	19.0	72.8	97253	77323	2401
2	2021-01-03	Maharashtra	19.0	72.8	94512	85607	1292
3	2021-01-04	Maharashtra	19.0	72.8	90460	75926	2175
4	2021-01-05	Maharashtra	19.0	72.8	99108	90351	1761

TOP 10 RECORDS BY CONFIRMED CASES

	Date	State	Latitude	Longitude	Confirmed	Recovered	Deaths
10	2021-01-11	Maharashtra	19.0	72.8	120800	112924	1961
47	2021-02-17	Maharashtra	19.0	72.8	113457	105521	2205
56	2021-02-26	Maharashtra	19.0	72.8	112468	92960	2103
53	2021-02-23	Maharashtra	19.0	72.8	111893	103675	1678
13	2021-01-14	Maharashtra	19.0	72.8	109127	92752	2491

6	2021-01-07	Maharashtra	19.0	72.8	108591	95675	2373
9	2021-01-10	Maharashtra	19.0	72.8	108412	87238	2264
50	2021-02-20	Maharashtra	19.0	72.8	108040	99763	1493
16	2021-01-17	Maharashtra	19.0	72.8	105687	84799	1543
57	2021-02-27	Maharashtra	19.0	72.8	105342	97943	1490

STATE-WISE SUMMARY

	Confirmed	Recovered	Deaths
State			
Andhra Pradesh	2552157	2211711	48295
Delhi	4501483	3889001	83207
Gujarat	2852362	2440200	55107
Karnataka	5339005	4563686	100284

Kerala	4927369	4241545	91273
Maharashtra	7599657	6531191	134934
Rajasthan	2375093	2026797	44086
Tamil Nadu	5975703	5135186	113679
Uttar Pradesh	3572031	3076210	65782
West Bengal	2663186	2278207	50122

TOP STATES BY CONFIRMED CASES

	Confirmed	Recovered	Deaths
State			
Maharashtra	7599657	6531191	134934
Tamil Nadu	5975703	5135186	113679
Karnataka	5339005	4563686	100284
Kerala	4927369	4241545	91273

Delhi	4501483	3889001	83207
Uttar Pradesh	3572031	3076210	65782
Gujarat	2852362	2440200	55107
West Bengal	2663186	2278207	50122
Andhra Pradesh	2552157	2211711	48295
Rajasthan	2375093	2026797	44086

CORRELATION MATRIX

	Confirmed	Recovered	Deaths
Confirmed	1.000000	0.991223	0.889182
Recovered	0.991223	1.000000	0.880920
Deaths	0.889182	0.880920	1.000000

OUTLIER LIMITS

Q1: 30864.0

Q3: 61599.75

IQR: 30735.75

Lower Limit: -15239.625

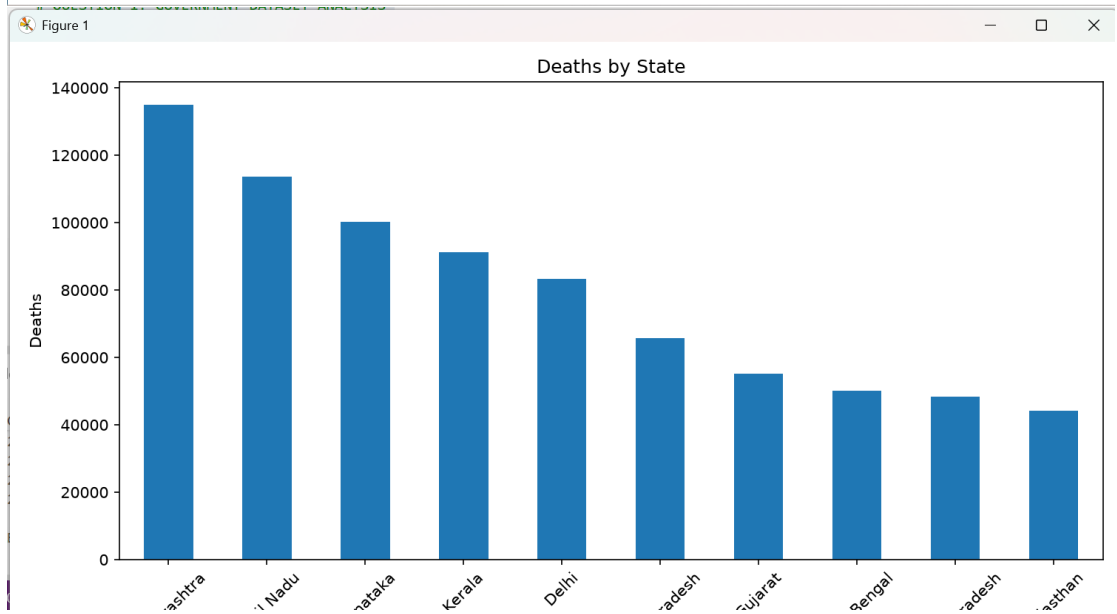
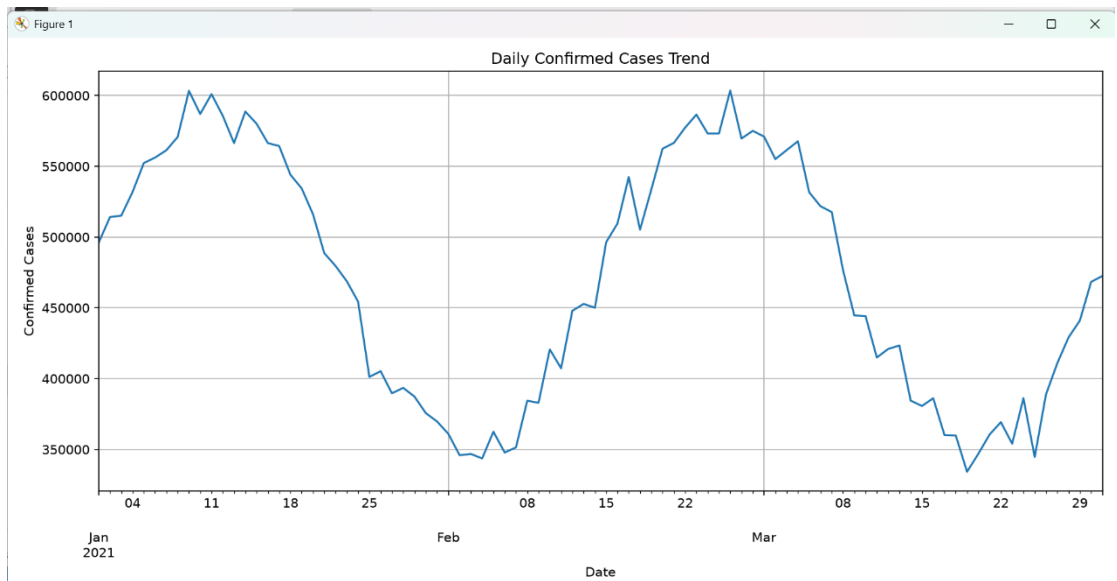
Upper Limit: 107703.375

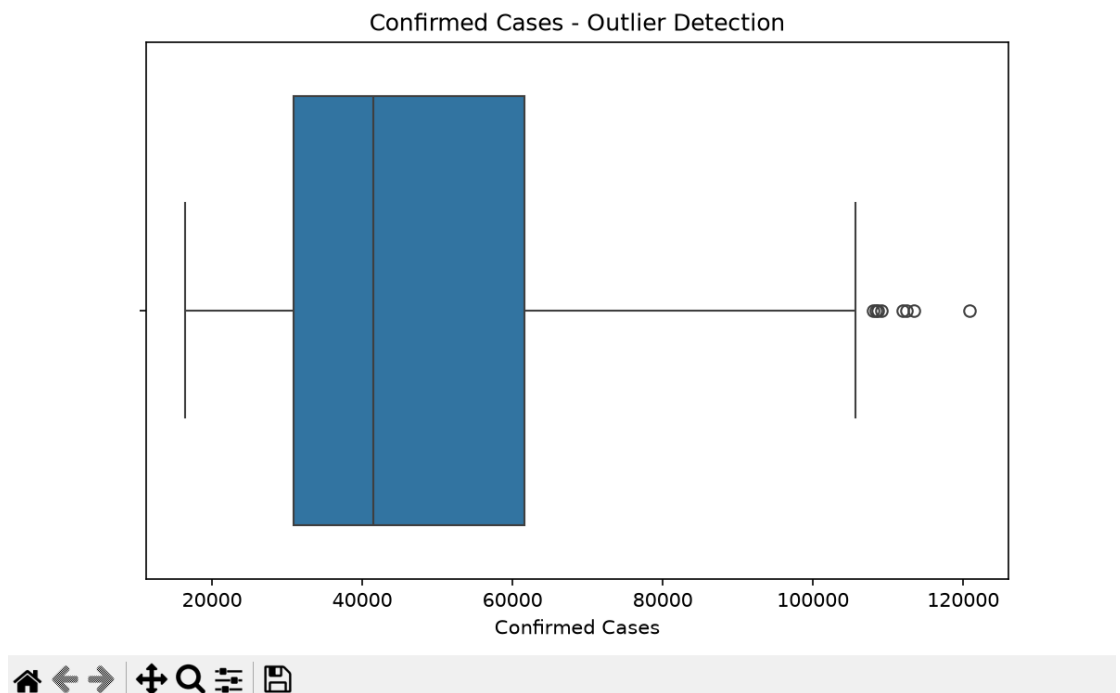
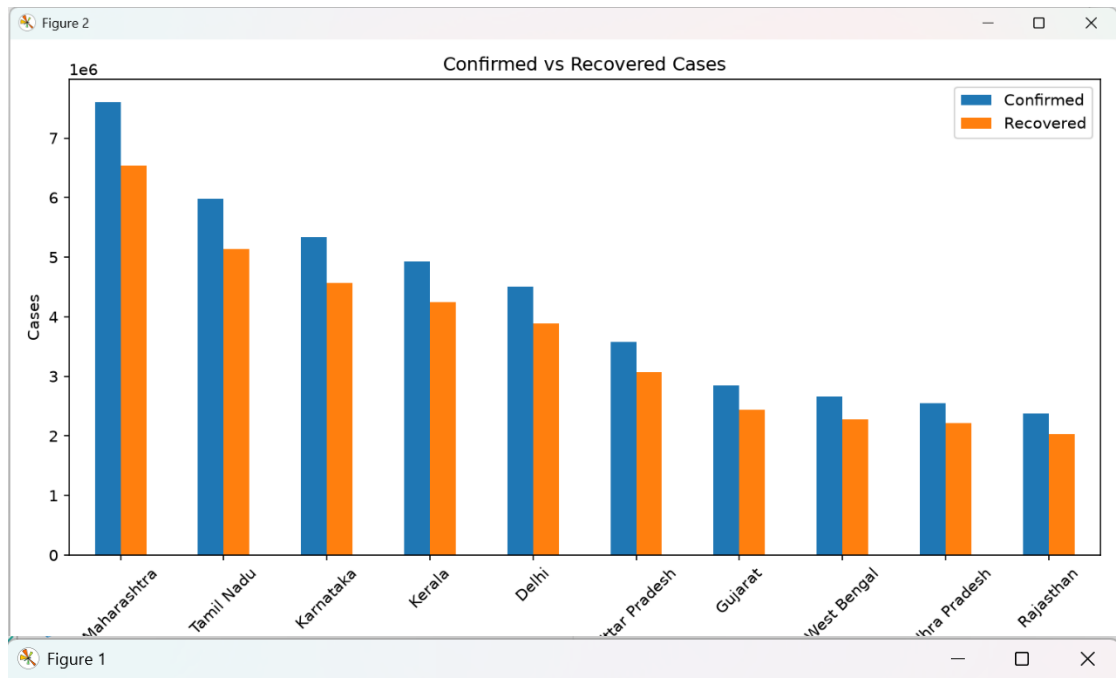
OUTLIER RECORDS

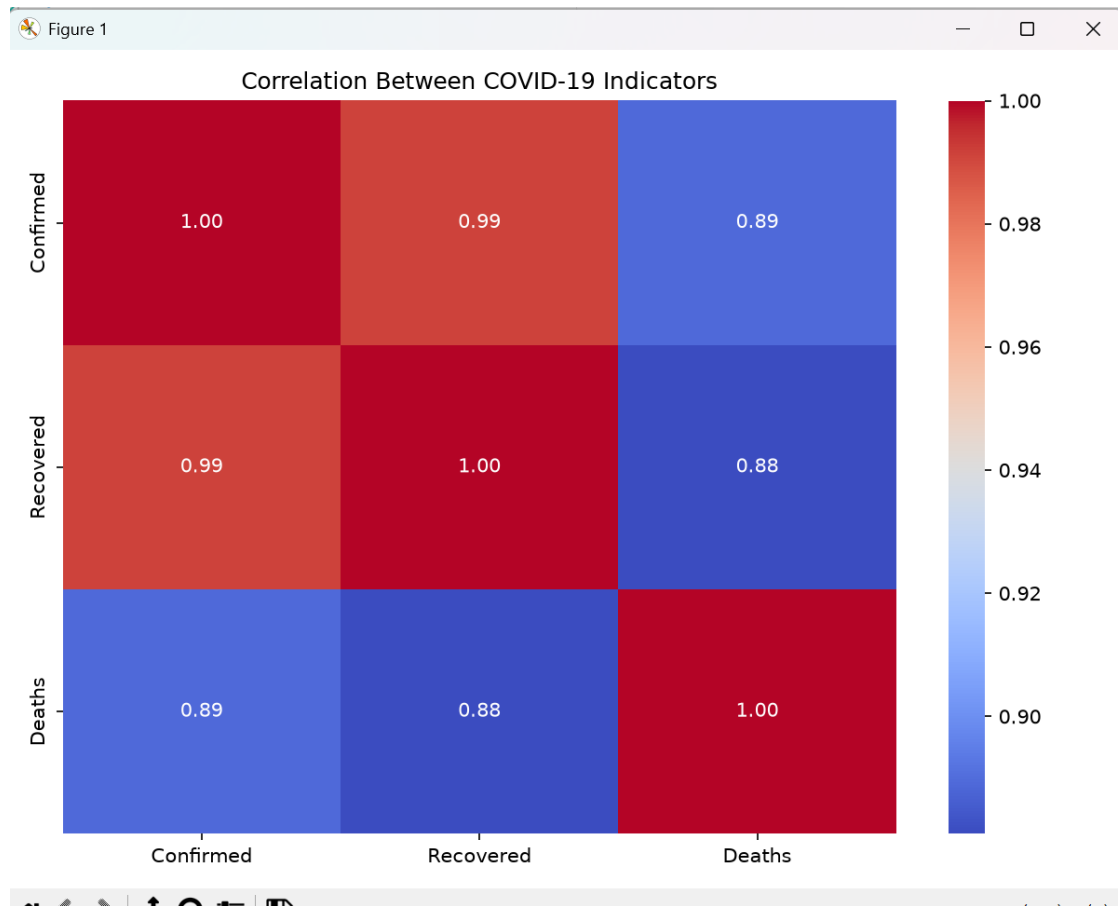
	Date	State	Latitude	Longitude	Confirmed	Recovered	Deaths
6	2021-01-07	Maharashtra	19.0	72.8	108591	95675	2373
9	2021-01-10	Maharashtra	19.0	72.8	108412	87238	2264
10	2021-01-11	Maharashtra	19.0	72.8	120800	112924	1961
13	2021-01-14	Maharashtra	19.0	72.8	109127	92752	2491
47	2021-02-17	Maharashtra	19.0	72.8	113457	105521	2205

47	2021-02-17	Maharashtra	19.0	72.8	113457	105521	2205
50	2021-02-20	Maharashtra	19.0	72.8	108040	99763	1493
53	2021-02-23	Maharashtra	19.0	72.8	111893	103675	1678
56	2021-02-26	Maharashtra	19.0	72.8	112468	92960	2103

NUMBER OF OUTLIERS: 8







2. Select any **open environmental dataset** such as air quality, rainfall, temperature, or pollution data. Analyze time-related patterns, identify correlations and outliers, create geographical visualizations/maps, and present the findings using an interactive visualization tool.

```

1  # COMMAND 2: Import libraries
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6  # COMMAND 3: Import Excel dataset
7  df = pd.read_excel(r"C:\Users\DELL\Downloads\Q2_Environmental_Air_Quality_Dataset.xlsx")
8
9  # COMMAND 4: Display first 5 records
10 print("FIRST 5 RECORDS")
11 print(df.head())
12 # COMMAND 5: Display last 5 records
13 print("\nLAST 5 RECORDS")
14 print(df.tail())
15 # COMMAND 6: Display dataset shape
16 print("\nDATASET SHAPE")
17 print(df.shape)

```

```

18 # COMMAND 7: Display column names
19 print("\nCOLUMN NAMES")
20 print(df.columns.tolist())
21 # COMMAND 8: Display dataset information
22 print("\nDATASET INFORMATION")
23 print(df.info())
24 # COMMAND 9: Statistical summary
25 print("\nSTATISTICAL SUMMARY")
26 print(df.describe())
27 # COMMAND 10: Check missing values
28 print("\nMISSING VALUES")
29 print(df.isnull().sum())
30 # COMMAND 11: Check duplicate records
31 print("\nDUPLICATE RECORDS")
32 print(df.duplicated().sum())
33 # COMMAND 12: Convert Date column
34 df["Date"] = pd.to_datetime(

```

```

35     df["Date"],
36     errors="coerce"
37 )
38 # COMMAND 13: Create Year and Month columns
39 df["Year"] = df["Date"].dt.year
40 df["Month"] = df["Date"].dt.month
41 print("\nDATA WITH YEAR AND MONTH")
42 print(df.head())
43 # COMMAND 14: Display unique cities
44 print("\nUNIQUE CITIES")
45 print(df["City"].unique())
46 # COMMAND 15: Filter high PM2.5 records
47 print("\nRECORDS WITH PM2.5 > 150")
48 high_pollution = df[
49     df["PM2.5"] > 150
50 ]
51 print(high_pollution.head(20))

```

```

51 print(high_pollution.head(20))
52 # COMMAND 16: Sort by PM2.5
53 print("\nTOP 10 HIGHEST PM2.5 RECORDS")
54 sorted_pm25 = df.sort_values(
55     by="PM2.5",
56     ascending=False
57 )
58 print(sorted_pm25.head(10))
59 # COMMAND 17: Group by city
60 print("\nCITY-WISE POLLUTION SUMMARY")
61 city_summary = df.groupby("City").agg(
62     PM2_5="mean",
63     PM10="mean",
64     NO2="mean",
65     SO2="mean",
66     CO="mean"
67 )

```

```

68 print(city_summary)
69 # COMMAND 18: Sort cities by PM2.5
70 print("\nCITIES SORTED BY PM2.5")
71 city_summary = city_summary.sort_values(
72     by="PM2.5",
73     ascending=False
74 )
75 print(city_summary)
76 # COMMAND 19: Monthly PM2.5 analysis
77 print("\nMONTHLY PM2.5")
78 monthly_pm25 = df.groupby(
79     "Month"
80 )[["PM2.5"]].mean()
81 print(monthly_pm25)
82 # COMMAND 20: Monthly PM2.5 line chart
83 plt.figure(figsize=(10, 6))
84 monthly_pm25.plot(

```

```

85     markers="o"
86 )
87 plt.title("Monthly Average PM2.5")
88 plt.xlabel("Month")
89 plt.ylabel("Average PM2.5")
90 plt.grid()
91 plt.tight_layout()
92 plt.show()
93 # COMMAND 21: City-wise PM2.5 chart
94 top10_cities = city_summary.head(10)
95 plt.figure(figsize=(10, 6))
96 top10_cities["PM2.5"].plot(
97     kind="bar"
98 )
99 plt.title("Average PM2.5 by City")
100 plt.xlabel("City")
101 plt.ylabel("Average PM2.5")

```

```

102 plt.xticks(rotation=45)
103 plt.tight_layout()
104 plt.show()
105 # COMMAND 22: Correlation analysis
106 print("\nCORRELATION MATRIX")
107 pollutants = [
108     "PM2.5",
109     "PM10",
110     "NO2",
111     "SO2",
112     "CO"
113 ]
114 correlation = df[
115     pollutants
116 ].corr()
117 print(correlation)
118 # COMMAND 23: Correlation heatmap

```

```

118 # COMMAND 23: Correlation heatmap
119 plt.figure(figsize=(10, 7))
120 sns.heatmap(
121     correlation,
122     annot=True,
123     cmap="coolwarm",
124     fmt=".2f"
125 )
126 plt.title("Correlation Between Air Pollutants")
127 plt.tight_layout()
128 plt.show()
129 # COMMAND 24: PM2.5 outlier detection
130 Q1 = df["PM2.5"].quantile(0.25)
131 Q3 = df["PM2.5"].quantile(0.75)
132 IQR = Q3 - Q1
133 lower_limit = Q1 - 1.5 * IQR
134 upper_limit = Q3 + 1.5 * IQR

```

```

135 print("\nPM2.5 OUTLIER LIMITS")
136 print("Q1:", Q1)
137 print("Q3:", Q3)
138 print("IQR:", IQR)
139 print("Lower Limit:", lower_limit)
140 print("Upper Limit:", upper_limit)
141 # COMMAND 25: Identify PM2.5 outliers
142 outliers = df[
143     (df["PM2.5"] < lower_limit) |
144     (df["PM2.5"] > upper_limit)
145 ]
146 print("\nPM2.5 OUTLIERS")
147 print(outliers)
148 print(
149     "\nNUMBER OF PM2.5 OUTLIERS:",
150     len(outliers)
151 )

```

```

152 # COMMAND 26: PM2.5 boxplot
153 plt.figure(figsize=(8, 5))
154 sns.boxplot(
155     x=df["PM2.5"]
156 )
157 plt.title("PM2.5 Outlier Detection")
158 plt.xlabel("PM2.5")
159 plt.show()
160 # COMMAND 27: Daily PM2.5 trend
161 daily_pm25 = df.groupby(
162     "Date"
163 )[["PM2.5"]].mean()
164 plt.figure(figsize=(12, 6))
165 daily_pm25.plot()
166 plt.title("Daily PM2.5 Trend")
167 plt.xlabel("Date")
168 plt.ylabel("Average PM2.5")

```

```

169 plt.grid()
170 plt.tight_layout()
171 plt.show()
172 # COMMAND 28: PM2.5 vs PM10 comparison
173 plt.figure(figsize=(12, 6))
174 city_summary[
175     ["PM2.5", "PM10"]
176 ].plot(
177     kind="bar",
178     figsize=(12, 6)
179 )
180 plt.title("PM2.5 and PM10 Comparison by City")
181 plt.xlabel("City")
182 plt.ylabel("Pollution Level")
183 plt.xticks(rotation=45)
184 plt.tight_layout()
185 plt.show()

```

```

186 # COMMAND 29: Create geographical map
187 city_locations = df.groupby(
188     ["City", "Latitude", "Longitude"]
189 )[["PM2.5"].mean().reset_index()
190 # COMMAND 30: Add cities to map
191 for _, row in city_locations.iterrows():
192     folium.CircleMarker(
193         location=[
194             row["Latitude"],
195             row["Longitude"]
196         ],
197         radius=8,
198         popup=(
199             f"City: {row['City']}<br>"
200             f"Average PM2.5: {row['PM2.5']:.2f}"
201         ),
202         fill=True
203     ).add_to(m)

```

```

203     ).add_to(m)
204 # COMMAND 31: Display interactive map
205 m
206 # COMMAND 32: Save interactive map
207 m.save(
208     "Q2_Air_Quality_Interactive_Map.html"
209 )
210 # COMMAND 33: Save city-wise analysis
211 city_summary.to_excel(
212     "Q2_City_Air_Quality_Analysis.xlsx"
213 )
214 # COMMAND 34: Save outliers
215 outliers.to_excel(
216     "Q2_Air_Quality_Outliers.xlsx",
217     index=False
218 )

```

FIRST 5 RECORDS

	Date	City	Latitude	Longitude	PM2.5	PM10	NO2	SO2	CO
0	2024-01-01	Delhi	28.6139	77.209	199.76	301.34	83.29	25.34	6.03
1	2024-01-02	Delhi	28.6139	77.209	195.04	348.59	58.59	31.43	5.78
2	2024-01-03	Delhi	28.6139	77.209	208.90	370.89	108.18	10.53	3.65
3	2024-01-04	Delhi	28.6139	77.209	216.08	374.66	71.66	25.72	3.14
4	2024-01-05	Delhi	28.6139	77.209	212.47	423.82	89.79	13.59	6.61

LAST 5 RECORDS

	Date	City	Latitude	Longitude	PM2.5	PM10	NO2	SO2	CO
3645	2024-12-26	Pune	18.5204	73.8567	82.80	169.39	36.84	5.26	2.08
3646	2024-12-27	Pune	18.5204	73.8567	91.88	150.50	34.96	12.07	1.87
3647	2024-12-28	Pune	18.5204	73.8567	92.31	149.38	44.45	15.39	1.96
3648	2024-12-29	Pune	18.5204	73.8567	95.33	153.24	29.41	12.37	1.49
3649	2024-12-30	Pune	18.5204	73.8567	84.61	163.42	22.59	8.77	2.66

DATASET SHAPE

(3650, 9)

COLUMN NAMES

['Date', 'City', 'Latitude', 'Longitude', 'PM2.5', 'PM10', 'NO2', 'SO2', 'CO']

DATASET INFORMATION

```

<class 'pandas.DataFrame'>
RangeIndex: 3650 entries, 0 to 3649
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        3650 non-null   datetime64[us]

```

1	City	3650	non-null	str
2	Latitude	3650	non-null	float64
3	Longitude	3650	non-null	float64
4	PM2.5	3650	non-null	float64
5	PM10	3650	non-null	float64
6	NO2	3650	non-null	float64
7	SO2	3650	non-null	float64

```
PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code>
8 CO 3650 non-null float64
dtypes: datetime64[us](1), float64(7), str(1)
memory usage: 256.8 KB
None
```

STATISTICAL SUMMARY									
	Date	Latitude	Longitude	PM2.5	PM10	NO2	SO2	CO	
count	3650	3650.000000	3650.000000	3650.000000	3650.000000	3650.000000	3650.000000	3650.000000	
mean	2024-07-01 00:00:00	20.900380	77.796420	87.078567	159.363545	34.443027	10.836855	1.940479	
min	2024-01-01 00:00:00	12.971600	72.571400	5.000000	10.000000	5.000000	2.000000	0.100000	
25%	2024-04-01 00:00:00	17.385000	73.856700	54.082500	99.105000	20.585000	5.722500	1.080000	
50%	2024-07-01 00:00:00	20.824300	77.401800	78.115000	141.800000	30.615000	9.125000	1.650000	
75%	2024-09-30 00:00:00	26.846700	80.270700	110.555000	204.032500	44.487500	14.227500	2.540000	
max	2024-12-30 00:00:00	28.613900	88.363900	658.122429	478.670000	126.070000	44.470000	7.930000	
std		NaN	5.326455	4.468606	47.550406	83.097863	18.945466	6.848432	1.211494
MISSING VALUES									
Date	0								
City	0								
Latitude	0								
Longitude	0								
PM2.5	0								
PM10	0								
NO2	0								
SO2	0								
CO	0								

dtype: int64											
DUPLICATE RECORDS											
0											
DATA WITH YEAR AND MONTH											
	Date	City	Latitude	Longitude	PM2.5	PM10	NO2	SO2	CO	Year	Month
0	2024-01-01	Delhi	28.6139	77.209	199.76	301.34	83.29	25.34	6.03	2024	1
1	2024-01-02	Delhi	28.6139	77.209	195.04	348.59	58.59	31.43	5.78	2024	1
2	2024-01-03	Delhi	28.6139	77.209	208.90	370.89	108.18	10.53	3.65	2024	1
3	2024-01-04	Delhi	28.6139	77.209	216.08	374.66	71.66	25.72	3.14	2024	1
4	2024-01-05	Delhi	28.6139	77.209	212.47	423.82	89.79	13.59	6.61	2024	1

UNIQUE CITIES											
<StringArray>											
['Delhi', 'Mumbai', 'Chennai', 'Bengaluru', 'Kolkata', 'Hyderabad', 'Ahmedabad', 'Jaipur', 'Lucknow', 'Pune']											
Length: 10, dtype: str											
RECORDS WITH PM2.5 > 150											
RECORDS WITH PM2.5 > 150											
	Date	City	Latitude	Longitude	PM2.5	PM10	NO2	SO2	CO	Year	Month
0	2024-01-01	Delhi	28.6139	77.209	199.76	301.34	83.29	25.34	6.03	2024	1
1	2024-01-02	Delhi	28.6139	77.209	195.04	348.59	58.59	31.43	5.78	2024	1
2	2024-01-03	Delhi	28.6139	77.209	208.90	370.89	108.18	10.53	3.65	2024	1
3	2024-01-04	Delhi	28.6139	77.209	216.08	374.66	71.66	25.72	3.14	2024	1
4	2024-01-05	Delhi	28.6139	77.209	212.47	423.82	89.79	13.59	6.61	2024	1

5	2024-01-06	Delhi	28.6139	77.209	223.16	454.21	79.93	37.36	3.38	2024	1
6	2024-01-07	Delhi	28.6139	77.209	210.32	388.65	54.66	17.62	5.91	2024	1
7	2024-01-08	Delhi	28.6139	77.209	193.51	301.21	99.58	32.82	6.14	2024	1
8	2024-01-09	Delhi	28.6139	77.209	220.54	434.88	89.87	32.62	7.27	2024	1
9	2024-01-10	Delhi	28.6139	77.209	219.57	377.89	59.67	42.10	4.64	2024	1
10	2024-01-11	Delhi	28.6139	77.209	207.35	433.12	113.48	15.40	6.22	2024	1
11	2024-01-12	Delhi	28.6139	77.209	209.02	320.69	99.54	21.57	5.48	2024	1

12	2024-01-13	Delhi	28.6139	77.209	210.59	354.17	61.19	14.22	2.65	2024	1
13	2024-01-14	Delhi	28.6139	77.209	212.01	370.17	66.78	32.60	5.76	2024	1
14	2024-01-15	Delhi	28.6139	77.209	205.56	442.87	75.16	36.05	2.44	2024	1
15	2024-01-16	Delhi	28.6139	77.209	196.06	429.75	98.81	26.85	5.03	2024	1
16	2024-01-17	Delhi	28.6139	77.209	205.33	405.66	98.79	41.01	4.58	2024	1
17	2024-01-18	Delhi	28.6139	77.209	199.98	383.37	57.13	17.34	3.64	2024	1
18	2024-01-19	Delhi	28.6139	77.209	224.16	365.08	67.19	12.72	7.14	2024	1

TOP 10 HIGHEST PM2.5 RECORDS											
	Date	City	Latitude	Longitude	PM2.5	PM10	NO2	SO2	CO	Year	Month
341	2024-12-07	Delhi	28.6139	77.2090	658.122429	413.98	71.90	27.12	4.81	2024	12
2566	2024-01-12	Jaipur	26.9124	75.7873	643.560192	349.49	92.64	13.89	2.63	2024	1
3253	2024-11-29	Lucknow	26.8467	80.9462	614.868095	345.26	65.66	20.20	5.46	2024	11
2619	2024-03-05	Jaipur	26.9124	75.7873	427.714363	254.38	47.58	24.55	3.81	2024	3
1792	2024-11-28	Kolkata	22.5726	88.3639	421.505296	230.61	35.26	20.71	2.25	2024	11

3300	2024-01-16	Pune	18.5204	73.8567	391.275925	220.44	55.91	9.94	1.57	2024	1
3379	2024-04-04	Pune	18.5204	73.8567	325.558495	148.35	41.44	13.21	1.05	2024	4
1637	2024-06-26	Kolkata	22.5726	88.3639	237.783702	120.80	36.31	7.14	1.98	2024	6
35	2024-02-05	Delhi	28.6139	77.2090	230.880000	444.96	126.07	14.61	7.93	2024	2
18	2024-01-19	Delhi	28.6139	77.2090	224.160000	365.08	67.19	12.72	7.14	2024	1

CITY-WISE POLLUTION SUMMARY

	PM2.5	PM10	NO2	SO2	CO
City					
Ahmedabad	95.776384	179.717452	38.680685	12.336603	2.138438
Bengaluru	48.564630	89.580137	19.574000	6.144247	1.123671
Chennai	55.253627	101.803041	22.103945	6.974356	1.241205

PS C:\Users\DELL\AppData\Local\Programs\Microsoft VS Code> & C:\Users\DELL\AppData\Local\Programs\Python\Python					
Delhi	146.168059	267.723233	56.979178	17.985178	3.177973
Hyderabad	60.013985	109.177890	23.748493	7.391014	1.261671
Jaipur	107.956235	195.848219	41.564575	13.053123	2.502301
Kolkata	86.959038	156.630658	34.442849	10.873041	1.921836
Lucknow	126.923337	232.334767	50.051836	16.123644	2.871096
Mumbai	75.259973	138.079370	30.435562	9.119699	1.661753
Pune	67.910404	122.740685	26.849151	8.367644	1.504849

CITIES SORTED BY PM2.5

	PM2.5	PM10	NO2	SO2	CO
City					
Delhi	146.168059	267.723233	56.979178	17.985178	3.177973
Lucknow	126.923337	232.334767	50.051836	16.123644	2.871096
Jaipur	107.956235	195.848219	41.564575	13.053123	2.502301
Ahmedabad	95.776384	179.717452	38.680685	12.336603	2.138438

Kolkata	86.959038	156.630658	34.442849	10.873041	1.921836
Mumbai	75.259973	138.079370	30.435562	9.119699	1.661753
Pune	67.910404	122.740685	26.849151	8.367644	1.504849
Hyderabad	60.013985	109.177890	23.748493	7.391014	1.261671
Chennai	55.253627	101.803041	22.103945	6.974356	1.241205
Bengaluru	48.564630	89.580137	19.574000	6.144247	1.123671

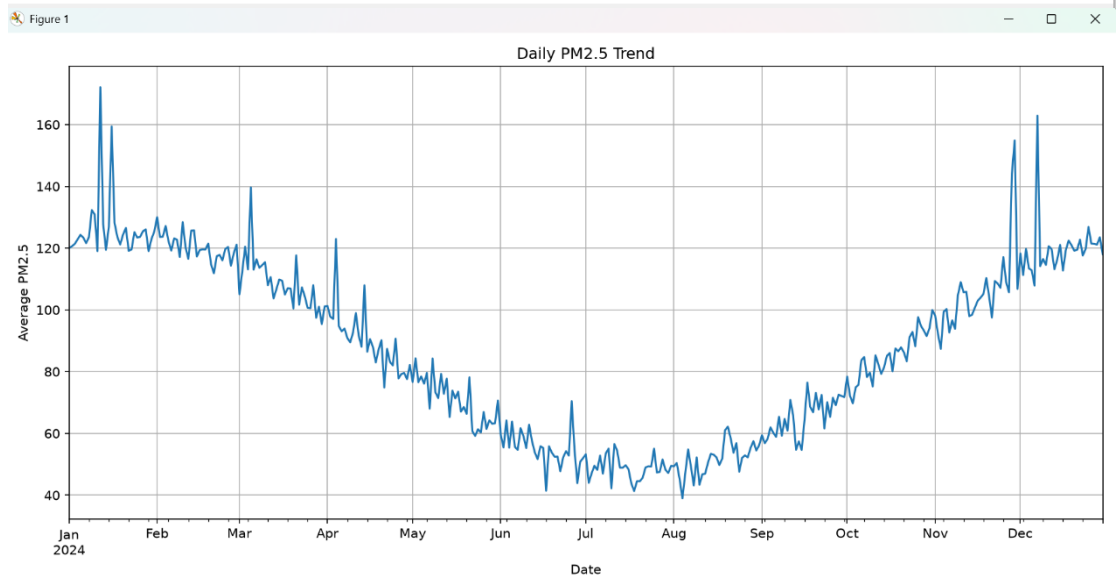
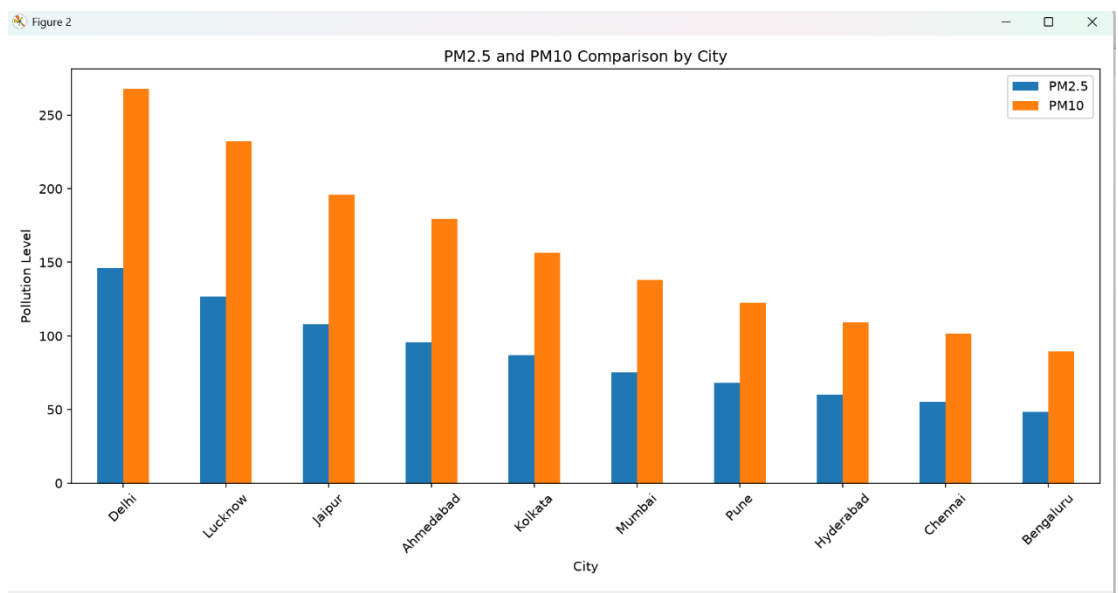
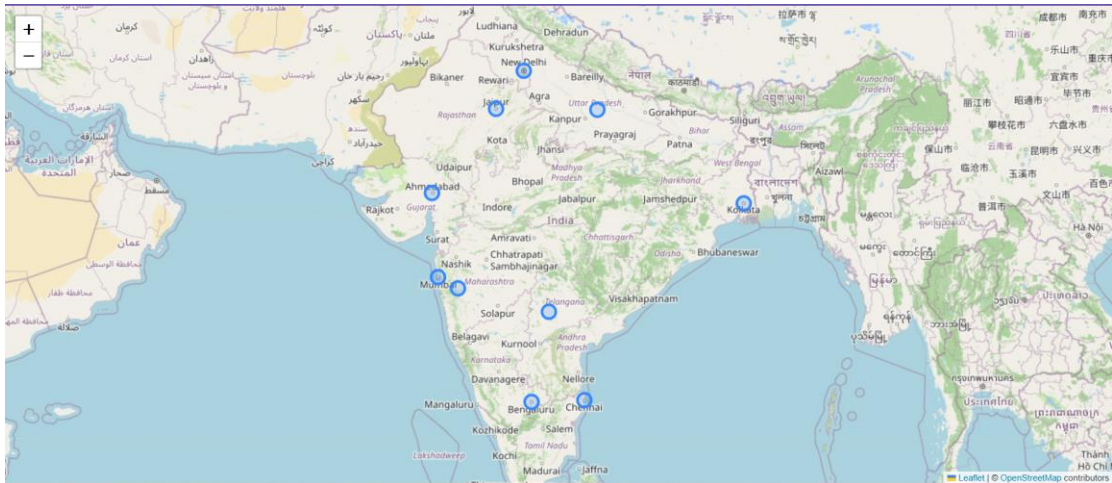


Figure 1

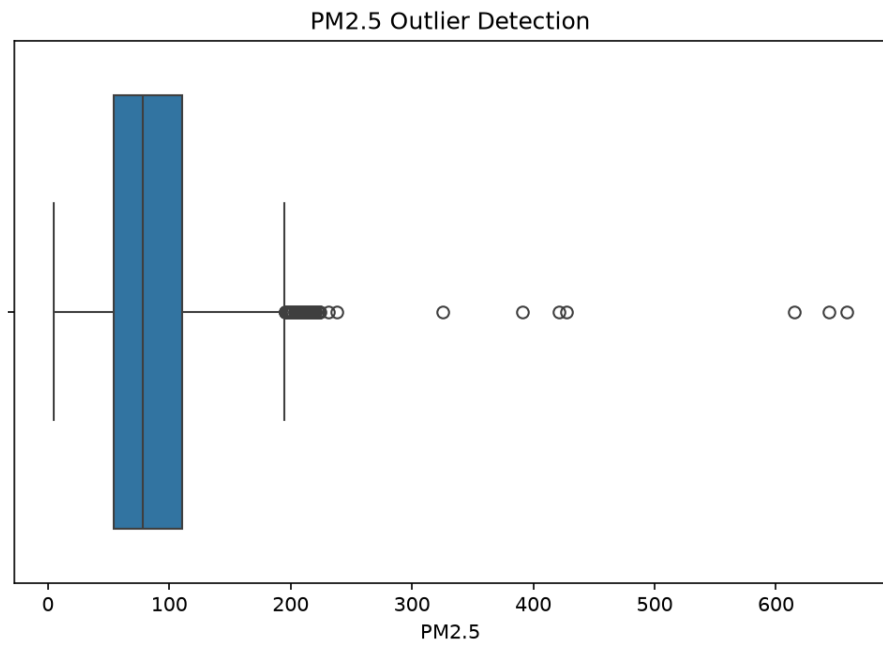


Figure 1

